MODULE 1-INTRODUCTION TO ARTIFICIAL INTELLIGENCE & PYTHON

DAY 1 - INTRODUCTION TO AI, MACHINE LEARNING AND DEEP LEARNING

1.WHAT IS ARTIFICIAL INTELLIGENCE?

Artificial Intelligence (AI) is a technology that enables computers and machines to perform tasks that normally require human intelligence.

AI systems can learn, reason, understand information, recognize patterns, and make decisions.

Examples:

- Chatbots and virtual assistants
- Face recognition
- Voice assistants
- Recommendation systems
- Self-driving technologies

Example of AI:

Google Maps uses AI-related technologies to understand traffic conditions and suggest routes.

Another simple example is **ChatGPT**. It can understand your question and generate a response.

2.WHAT CAN AI DO ?

AI can perform many types of intelligent tasks**.**

Learning

AI systems can learn patterns from information.

Example:
A system can learn which emails are usually spam.

Perception

AI can understand images, sounds, and other inputs.

Example:
Face recognition can identify a person from an image.

Language Understanding

AI can process human language.

Example:
Chatbots can understand questions and respond.

Problem Solving

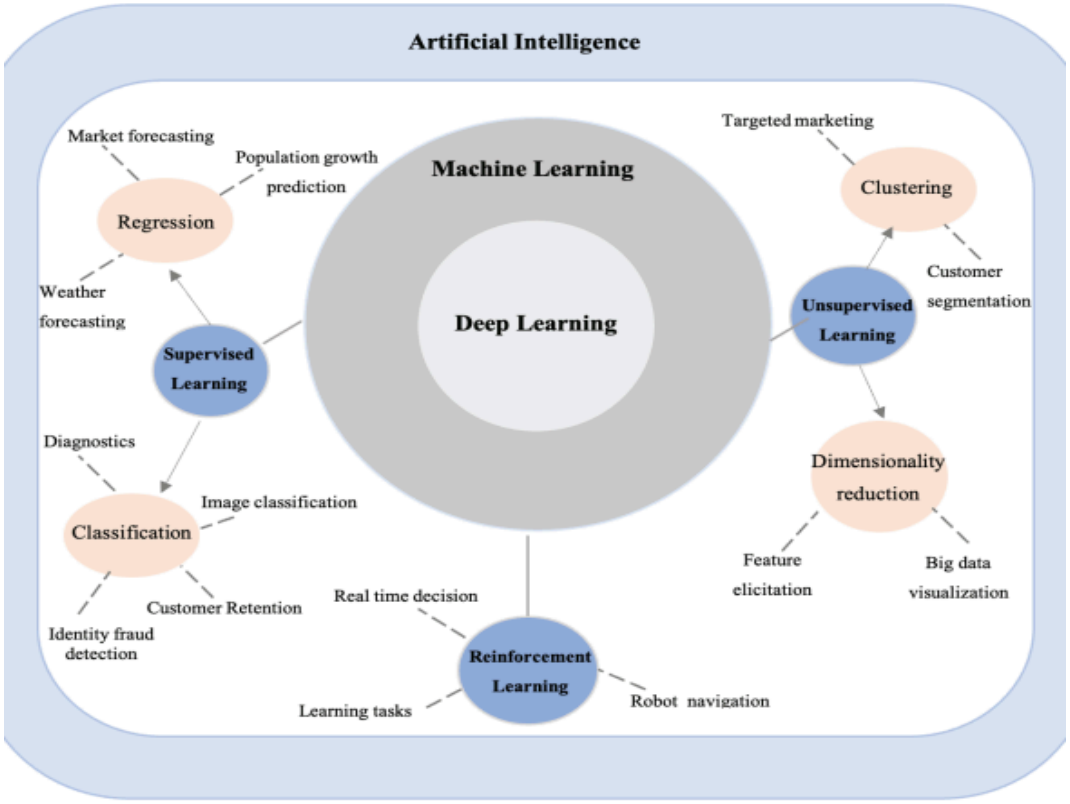
AI can find solutions to problems.

Example:
A navigation system can find a suitable route between two locations.

Decision Making

AI can use available information to make decisions or recommendations.

Example:
Netflix or YouTube can recommend content based on your activity.



3.WHAT IS MACHINE LEARNING ?

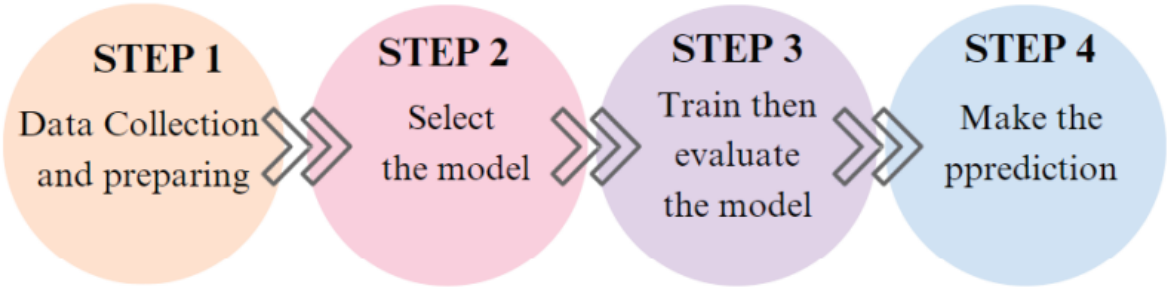
Machine Learning (ML) is a branch of Artificial Intelligence that allows computers to learn from data and improve their performance without being explicitly programmed for every task.It identifies patterns in data and uses those patterns to make predictions or decisions.

Example:

An email system can learn from previous emails and identify whether a new email is spam or not.

Common applications of ML:

- Spam detection
- Product recommendations
- Fraud detection
- Customer prediction



4.WHAT IS DEEP LEARNING ?

Deep Learning is a subset of Machine Learning that uses artificial neural networks with multiple layers to learn complex patterns from large amounts of data.

Deep Learning is especially useful for working with images, audio, video, and natural language.

Examples:

- Face recognition
- Image classification
- Speech recognition
- Self-driving systems
- Language translation

SIMPLE HIERARCHY:

Artificial Intelligence
I
Machine Learning
I
Deep Learning

5.Machine Learning: Types and Algorithms

Machine Learning is categorized by how the algorithm learns from data.

Types of Learning

Supervised Learning: The model learns from labeled data (input-output pairs). It is used for prediction and classification.

Unsupervised Learning: The model works with unlabeled data to find hidden patterns or structures.

Reinforcement Learning: The model learns by interacting with an environment and receiving rewards or penalties for its actions.

Semi-Supervised Learning: A hybrid approach that uses a small amount of labeled data and a large amount of unlabeled data to improve learning accuracy.

Common ML Algorithms

- **Linear Regression:** Used for predicting continuous values (e.g., house prices).
- **Logistic Regression:** Used for binary classification (e.g., yes/no, spam/not spam).
- **Decision Trees:** Rule-based models that split data into branches to reach a decision.
- **Random Forest:** An ensemble of decision trees that improves accuracy and prevents overfitting.
- **Support Vector Machines (SVM):** Used for complex classification tasks by finding the best boundary between data points.

6.DIFFERENCE BETWEEN AI,ML AND DEEP LEARNING

ARTIFICIAL INTELLIGENCE	MACHINE LEARNING	DEEP LEARNING
AI is the broader concept of making machines intelligent.	ML is a subset of AI that learns from data.	DL is a subset of ML that uses multi-layer neural networks.

It can use rules, logic, learning, and other techniques	It learns patterns from data.	It learns complex patterns from large datasets.
Example: Virtual assistant	Example: Spam detection	Example: Face recognition

---

7.REAL-WORLD APPLICATIONS OF AI

Artificial Intelligence is used in many areas of everyday life.

Healthcare: AI can help analyze medical images and support healthcare professionals.

Banking: AI and ML can be used for fraud detection and risk analysis.

E-commerce: Recommendation systems suggest products based on user activity.

Education: AI can provide personalized learning experiences.

Transportation: AI is used in navigation, traffic prediction, and advanced driver-assistance systems.

Entertainment: Streaming platforms use recommendation systems to suggest movies, music, and videos.

8.Deep Learning: Architectures and Models

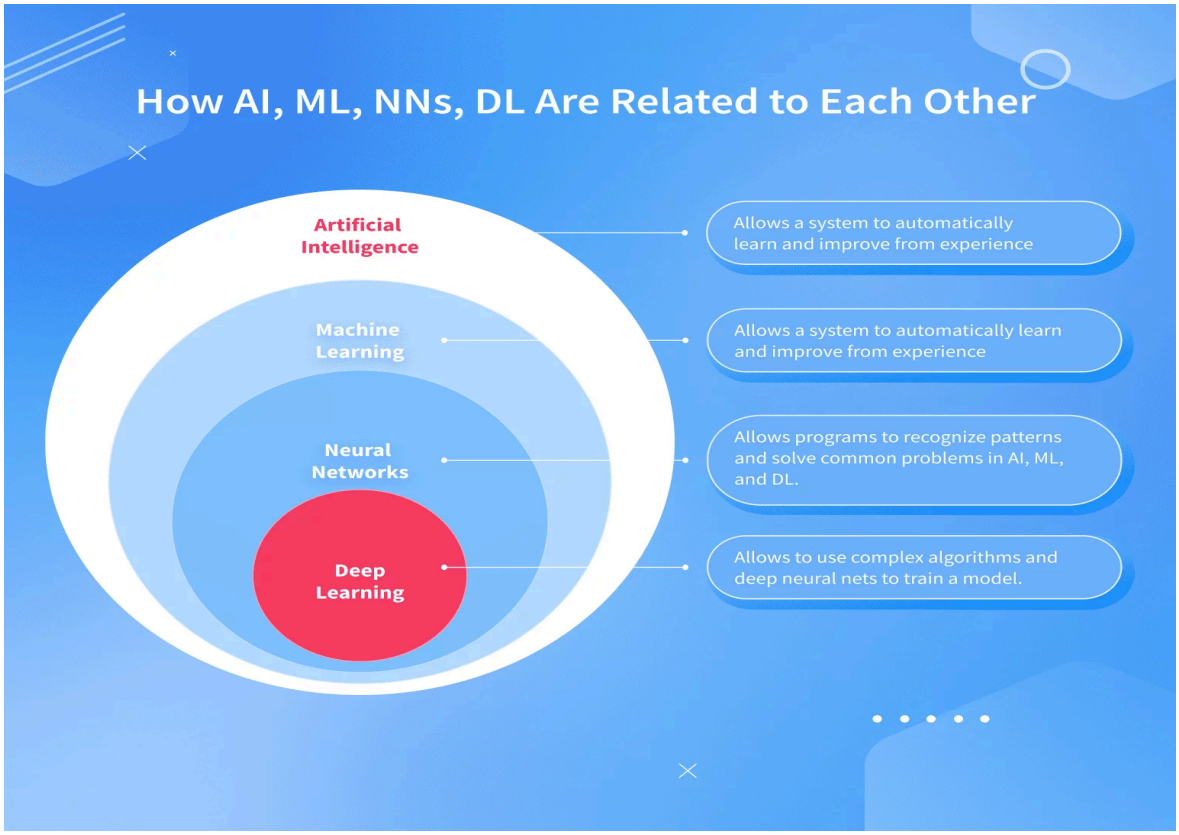
Deep Learning uses multi-layered neural networks to handle complex, unstructured data like images, audio, and text

Key Architectures

- **Artificial Neural Networks (ANN):** The foundational architecture inspired by biological neurons.
- **Convolutional Neural Networks (CNN):** Specialized for image processing and computer vision. They identify patterns like edges, shapes, and objects.
- **Recurrent Neural Networks (RNN):** Designed for sequential data like text or speech, where the order of information matters.
- **Transformers:** The modern standard for Natural Language Processing (NLP), powering models like GPT.

Key Technical Differences

- **Feature Engineering:** In traditional ML, humans must manually select and label features (e.g., "this is a tail"). In DL, the neural network automatically extracts these features from raw data
- **Data Volume:** ML performs well with small to medium structured datasets. DL requires massive amounts of unstructured data to be effective
- **Hardware:** ML can run on standard CPUs. DL often requires high-performance GPUs due to the massive number of calculations in its many layers
- **Interpretability:** ML models like decision trees are "white boxes" (easy to explain). DL models are often "black boxes," making it difficult to understand exactly how they reached a specific decision.



DAY 2- PYTHON BASICS

INTRODUCTION OF PYTHON

1.WHAT IS PYTHON?

Python is a high-level, interpreted programming language known for its simple and readable syntax.

Python is widely used in Artificial Intelligence, Machine Learning, Data Science, Web Development, Automation, and many other areas.It is popular in AI and ML because it has many useful libraries and frameworks.

Examples of Python libraries used in AI and ML:

- NumPy
- Pandas
- Matplotlib
- Scikit-learn
- TensorFlow
- PyTorch

2.VARIABLES WITH EXAMPLE

A variable is a name used to store a value in a program.

Example:

```
name = "ABCDE"

age = 21

course = "AI and Machine Learning"

print(name)

print(age)

print(course)
```

HERE:

- `name` stores a string value.

- `age` stores an integer value.
- `course` stores a string value

3.WHAT ARE THE DATA TYPES

A data type defines the type of value stored in a variable.

Common Python data types include:

String (`str`) – Used for text.

name = "ABCDE"

Integer (`int`) – Used for whole numbers.

age = 21

Float (`float`) – Used for decimal numbers.

price = 99.50

Boolean (`bool`) – Represents True or False.

is_student = True

List (`list`) – Stores multiple values.

languages = ["Python", "Java", "C++"]

4.WHAT IS OPERATORS WITH EX

Operators are symbols used to perform operations on values.

1. Arithmetic Operators

These operators are used to perform mathematical calculations.

Symbol	Operator	Explanation	Example	Result
`+`	Addition	Adds two values	`10 + 5`	`15`
`-`	Subtraction	Subtracts the second value from the first	`10 - 5`	`5`
`*`	Multiplication	Multiplies two values	`10 * 5`	`50`
`/`	Division	Divides the first value by the second	`10 / 5`	`2.0`
`%`	Modulus	Returns the remainder after division	`10 % 3`	`1`
`//`	Floor Division	Returns the whole-number part of division	`10 // 3`	`3`
`**`	Exponentiation	Raises a number to a power	`2 ** 3`	`8`

---

2. Comparison Operators

These operators compare two values. The result is either **True** or **False**.

Symbol	Operator	Explanation	Example	Result

`==`	Equal to	Checks whether two values are equal	`10 == 10`	`True`
`!=`	Not equal to	Checks whether two values are different	`10 != 5`	`True`
`>`	Greater than	Checks whether the first value is greater	`10 > 5`	`True`
`<`	Less than	Checks whether the first value is smaller	`10 < 5`	`False`
`>=`	Greater than or equal to	Checks whether the first value is greater than or equal to the second	`10 >= 10`	`True`
`<=`	Less than or equal to	Checks whether the first value is smaller than or equal to the second	`5 <= 10`	`True`

---

3. Assignment Operators

These operators are used to assign values to variables or update their values.

Symbol	Operator	Explanation	Example
`=`	Assignment	Assigns a value	`x = 10`
`+=`	Add and assign	Adds a value and assigns the result	`x += 5`

`-=`	Subtract and assign	Subtracts a value and assigns the result	`x -= 5`
`*=`	Multiply and assign	Multiplies and assigns the result	`x *= 5`
`/=`	Divide and assign	Divides and assigns the result	`x /= 5`
`//=`	Floor divide and assign	Performs floor division and assigns	`x //= 5`
`%=`	Modulus and assign	Finds remainder and assigns	`x %= 5`
`**=`	Exponentiate and assign	Performs exponentiation and assigns	`x **= 2`

Example:

x = 10

x += 5

print(x)

Output:

15

Here, `x += 5` means:

x = x + 5

---

4. Logical Operators

Logical operators are used to combine or reverse conditions.

Keyword	Operator	Explanation	Example	Result
`and`	AND	True only when both conditions are True	`10 > 5 and 10 < 20`	`True`
`or`	OR	True when at least one condition is True	`10 > 20 or 10 < 20`	`True`
`not`	NOT	Reverses the Boolean result	`not(10 > 5)`	`False`

---

5. Identity Operators

These operators check whether two variables refer to the same object.

Symbol	Explanation
`is`	Returns True if both refer to the same object
`is not`	Returns True if they do not refer to the same object

Example:

x = None

print(x is None)

Output:

True

---

6. Membership Operators

These operators check whether a value exists in a sequence such as a list or string.

Symbol	Explanation
`in`	Checks whether a value exists
`not in`	Checks whether a value does not exist

Example:

languages = ["Python", "Java", "C++"]

print("Python" in languages)

Output:

True

---

7. Bitwise Operators

These operators perform operations on the binary representation of numbers.

Symbol	Name	Explanation

&	Bitwise AND	Performs AND operation on bits
`	`	Bitwise OR
^	Bitwise XOR	Performs XOR operation on bits
~	Bitwise NOT	Inverts the bits
<<	Left Shift	Shifts bits to the left
>>	Right Shift	Shifts bits to the right

Example:

```
name = "ABCDE"
age = 21

print("Name:", name)
print("Age:", age)
```

Output:

```
Name: ABCDE
Age: 21
```

DAY 3 - PROGRAMMING CONCEPTS (Conditions, loops and lists)

IF CONCEPT:

Example:

```
marks = 75


if marks >= 50:
    print("Pass")
else:
    print("Fail")
```

FOR LOOP CONCEPT

```
for i in range(1, 6):
    print(i)
```

LIST CONCEPT

```
languages = ["Python", "Java", "C++"]


for language in languages:
    print(language)
```

DAY4 - FUNCTIONS AND BEGINNER PROGRAMS

FUNCTIONS:

Example:

```
def greet(name):
    print("Hello", name)


greet("Swetha")
```

EVEN / ODD NUMBER

EXAMPLE:

```
num = int(input("Enter a number: "))
if num % 2 == 0:
    print("Even number")
else:
    print("Odd number")
```

OUTPUT:

Enter a number: 8

Even number

Explanation:

- `input()` gets a number from the user.
- `int()` converts the input into an integer.
- `num % 2 == 0` checks whether the remainder is zero.
- If true, the number is even; otherwise, it is odd.

POSITIVE / NEGATIVE NUMBER

EXAMPLE:

```
num = int(input("Enter a number: "))

if num > 0:

    print("Positive number")

elif num < 0:

    print("Negative number")

else:

    print("Zero")
```

OUTPUT:

Enter a number: -5

Negative number

Explanation:

- `num > 0` means positive.
- `num < 0` means negative.
- Otherwise, the number is zero.

LARGEST TO TWO NUMBERS

EXAMPLE:

```
a = int(input("Enter first number: "))

b = int(input("Enter second number: "))

if a > b:

    print("Largest number is:", a)

elif b > a:
```

```
    print("Largest number is:", b)
else:
    print("Both numbers are equal")
```

OUTPUT:

Enter first number: 25

Enter second number: 40

Largest number is: 40

Explanation: The program compares both numbers and prints the larger one. If they are equal, it displays a separate message.

SUM OF NUMBERS

EXAMPLE:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
total = a + b

print("Sum =", total)
```

OUTPUT:

Enter first number: 10

Enter second number: 20

Sum = 30

Explanation: The values are stored in `a` and `b`. Their sum is saved in `total` and printed.

MULTIPLICATION TABLE

EXAMPLE:

```
num = int(input("Enter a number: "))


for i in range(1, 11):
    print(num, "x", i, "=", num * i)
```

OUTPUT:

Enter a number: 5

5 x 1 = 5

5 x 2 = 10

5 x 3 = 15

5 x 4 = 20

5 x 5 = 25

5 x 6 = 30

5 x 7 = 35

5 x 8 = 40

5 x 9 = 45

5 x 10 = 50

Explanation: `range(1, 11)` generates numbers from 1 through 10. Each iteration multiplies the entered number by `i`.

FACTORIAL OF A NUMBER

EXAMPLE:

```
num = int(input("Enter a non-negative integer: "))
```

```
if num < 0:
    print("Factorial is not defined for negative numbers")
else:
    factorial = 1
    for i in range(1, num + 1):
        factorial = factorial * i


    print("Factorial =", factorial)
```

OUTPUT:

Enter a non-negative integer: 5

Factorial = 120

Explanation:

- `factorial = 1` initializes the result.
- The loop multiplies the result by each integer up to `num`.
- For input `0`, the result is `1`, because 0!=10! = 10!=1.

REVERSE A STRING

EXAMPLE:

```
text = input("Enter a string: ")
reversed_text = text[::-1]
print("Reversed string:", reversed_text)
```

OUTPUT:

Enter a string: hello

Reversed string: olleh

Explanation: `[::-1]` reads the string backward, starting from the last character.

COUNT VOWELS

EXAMPLE:

```
text = input("Enter a string: ")

vowels = "aeiou"

count = 0

for char in text.lower():

    if char in vowels:

        count += 1


print("Number of vowels:", count)
```

OUTPUT:

Enter a string: Hello World

Number of vowels: 3

Explanation:

- `text.lower()` converts the text to lowercase.
- `char` represents each character.
- If the character is a vowel, `count += 1` increases the count by one.

FIND LARGEST NUMBER IN A LIST

EXAMPLE:

```
numbers = [10, 25, 7, 45, 18]
```

```
largest = numbers[0]

for num in numbers:

    if num > largest:

        largest = num


print("Largest number is:", largest)
```

OUTPUT:

Largest number is: 45

Explanation:

- `numbers[0]` takes the first number as the initial largest value.
- The loop compares each number with `largest`.
- When a larger number is found, `largest` is updated.

Note: This example assumes the list contains at least one number.

SIMPLE CALCULATOR

EXAMPLE:

```
a = float(input("Enter first number: "))

operator = input("Enter operator (+, -, *, /): ")

b = float(input("Enter second number: "))


if operator == "+":

    print("Result =", a + b)

elif operator == "-":

    print("Result =", a - b)
```

```
elif operator == "*":
    print("Result =", a * b)
elif operator == "/":
    if b != 0:
        print("Result =", a / b)
    else:
        print("Error: Cannot divide by zero")
else:
    print("Invalid operator")
```

OUTPUT:

Enter first number: 10

Enter operator (+, -, *, /): *

Enter second number: 5

Result = 50.0

Explanation:

- `float()` accepts both whole numbers and decimal numbers.
- The program checks which operator the user entered.
- It performs the matching calculation.
- Division by zero is handled safely.
